

Pearls AQI Predictor: An End-to-End Serverless Machine Learning System for Air Quality Forecasting in Karachi

Final Project Report

Ayesha Salahuddin
Pearls AQI Predictor Project
Karachi, Pakistan

Abstract—This report presents the complete design, implementation, and deployment of the Pearls AQI Predictor, an end-to-end and fully serverless machine learning system that forecasts the Air Quality Index (AQI) of Karachi for the next three days. The system automatically collects live and historical air quality and weather data, engineers predictive features and stores them in a managed feature store, trains and compares multiple forecasting models, automates the entire workflow through a continuous integration and deployment (CI/CD) platform, and serves predictions through a publicly deployed interactive dashboard that includes alerts for hazardous air quality. The implementation integrates the Hopworks Feature Store and Model Registry, GitHub Actions for scheduling, scikit-learn and PyTorch for modeling, and Streamlit for the user interface. Three models, Ridge Regression, Random Forest, and a Long Short-Term Memory (LSTM) network, were trained and evaluated using RMSE, MAE, and the coefficient of determination (R^2); Random Forest performed best and was promoted to production. Model behaviour was explained using SHAP. This report documents the methodology of each component in detail and provides an honest discussion of the engineering challenges encountered and the genuine limitations of multi-day air quality forecasting.

Index Terms—air quality index, machine learning, MLOps, feature store, serverless, time-series forecasting, Hopworks, Streamlit, CI/CD, SHAP, explainability

I. INTRODUCTION

Air pollution is a persistent and serious public-health problem in Karachi, where the Air Quality Index frequently rises to levels considered unhealthy. Timely forecasts of air quality can help individuals, particularly sensitive groups, plan their activities and reduce exposure. The objective of the Pearls AQI Predictor is to build a complete, automated system that forecasts Karachi's AQI three days into the future using a fully serverless technology stack, meaning that no part of the system requires a managed server or manual operation.

The project was developed across eight milestones, progressing from environment setup and data collection, through exploratory analysis and model training, to automation, deployment, and documentation. The guiding principle throughout was end-to-end automation: data should be collected on a schedule, the model should be retrained automatically as new data arrives, and predictions should be served continuously to the public without human intervention.

This report consolidates the entire project. It describes the system architecture, the data collection and feature engineering methodology, the exploratory analysis, the training and evaluation of three models, the explainability analysis, the automation layer, the deployed dashboard, and a candid assessment of limitations and future work.

II. SYSTEM ARCHITECTURE

The system adopts a standard machine-learning operations (MLOps) architecture comprising four components linked through a central feature store and model registry.

The **feature pipeline** fetches raw weather and pollutant data from external APIs, computes engineered features, and writes them to the Feature Store. The **training pipeline** reads historical features, trains and evaluates the candidate models, and registers the best model in the Model Registry. The **automation layer** schedules both pipelines using GitHub Actions. The **web application** loads the trained model and the most recent features and serves the three-day forecast through an interactive dashboard.

The Hopworks platform provides both the Feature Store and the Model Registry and acts as the central hub. This design decouples the components from one another: each pipeline can run independently and on its own schedule, communicating only through the stored features and models. A practical benefit of this separation, realised in this project, is that model experimentation can occur in a notebook while a separate automation script handles production training, without the two interfering.

III. DATA COLLECTION AND FEATURE ENGINEERING

A. Data Sources

The system targets Karachi and operates at an hourly time resolution. Two categories of variables are used: meteorological variables, including temperature, humidity, wind speed, and atmospheric pressure; and pollutant concentrations, including fine particulate matter (PM_{2.5}), coarse particulate matter (PM₁₀), nitrogen dioxide (NO₂), ozone (O₃), sulfur dioxide (SO₂), and carbon monoxide (CO).

Live data is obtained from the AQICN API, which provides current readings for Karachi. This source powers the hourly feature pipeline that keeps the system up to date.

B. Historical Backfill and a Data-Quality Correction

To train a model, a substantial history of observations was required. An initial backfill was attempted using the AQICN API. However, it was discovered that the free tier of AQICN returns only the most recent reading rather than genuine historical observations. The resulting dataset therefore contained an almost constant AQI value, which is unsuitable for training a forecasting model.

This problem was resolved by adopting the Open-Meteo Air Quality and Archive APIs for historical pollutant and weather data, which provide true hourly historical values. Because Open-Meteo returns pollutant concentrations rather than a pre-computed AQI, the AQI was derived from the PM2.5 concentration using the United States Environmental Protection Agency (US EPA) breakpoint formula:

$$AQI = \frac{I_{hi} - I_{lo}}{C_{hi} - C_{lo}}(C - C_{lo}) + I_{lo} \quad (1)$$

where C is the measured PM2.5 concentration, (C_{lo}, C_{hi}) are the concentration breakpoints of the band containing C , and (I_{lo}, I_{hi}) are the corresponding AQI breakpoints. Computing AQI from PM2.5 is a deliberate simplification: the official AQI is the maximum of sub-indices computed across several pollutants, but PM2.5 is consistently the dominant pollutant in Karachi, so it serves as a strong proxy. This correction produced a dataset of 4,344 hourly records beginning on 29 November 2025 and spanning approximately six months.

C. Feature Engineering

The predictive power of the system rests heavily on feature engineering, which transforms raw hourly readings into informative inputs. The following families of features were constructed, yielding 41 model features in total:

- **Lagged AQI values:** the AQI at offsets of 1, 2, 3, 6, 12, 24, 48, 71, 72, 73, 96, 120, and 168 hours earlier. The offsets clustered around 72 hours and the 168-hour (weekly) lag were chosen to give the model direct access to values near the forecast horizon and to weekly periodicity.
- **Rolling statistics:** moving averages of AQI over 24-hour, 72-hour, and 168-hour windows, and a 24-hour rolling standard deviation, capturing recent trend and volatility. All rolling features are shifted by one hour to avoid using the current value.
- **Lagged meteorological drivers:** PM2.5, wind, and temperature at 24-hour and 72-hour lags, reflecting that recent weather influences pollutant accumulation and dispersion.
- **Cyclical time encodings:** the hour of day and month of year each encoded as sine and cosine pairs, so that the model treats time as continuous and periodic.

- **Calendar features:** hour, day of week, month, and a weekend indicator.

The forecasting target is the AQI 24 hours into the future, created by shifting the AQI column. Rows containing missing values introduced by the lag and rolling operations were dropped, leaving a clean modeling dataset.

IV. EXPLORATORY DATA ANALYSIS

Exploratory analysis was performed before modeling. The dataset was complete, with no missing hourly records across the period. Several intuitive and useful patterns emerged. PM2.5 was almost perfectly correlated with AQI, consistent with the decision to derive AQI from PM2.5 and confirming PM2.5 as the dominant driver. A clear daily cycle was observed, with air quality worst in the early-morning hours and cleanest around midday, consistent with nighttime temperature inversions that trap pollutants near the ground. A seasonal pattern was evident, with the winter months substantially more polluted than the spring months. Wind speed and temperature showed negative associations with AQI, consistent with the physical dispersion of pollutants. Variation between weekdays and weekends was minimal. These findings directly informed the choice of lag, rolling, and meteorological features.

V. MODEL TRAINING AND EVALUATION

A. Experimental Setup

Three models were trained and compared, in keeping with the requirement to experiment with standard machine-learning models and an advanced deep-learning model. To obtain a realistic estimate of forecasting performance and to prevent leakage of future information, the data was split chronologically without shuffling: the earliest 80% of records were used for training and the most recent 20% for testing. Models were evaluated using the root mean squared error (RMSE), the mean absolute error (MAE), and the coefficient of determination (R^2).

An important part of the experimentation was the choice of forecast horizon. Initial experiments targeting AQI 72 hours ahead produced very weak results (Random Forest R^2 near 0.01), confirming that three-day-ahead values are difficult to predict directly. The horizon was therefore set to 24 hours ahead for the production model, with the three-day forecast generated recursively at serving time, as described later.

B. Ridge Regression

Ridge Regression is a regularized linear model that predicts AQI as a weighted sum of the input features, with an L2 penalty (regularization strength $\alpha = 1.0$) that discourages large coefficients and reduces overfitting. It serves as an interpretable linear baseline. Because the penalty is sensitive to feature scale, the inputs were standardized before fitting. Ridge is fast and stable but is limited by its assumption of linear relationships, which cannot capture the non-linear behaviour present in air-quality dynamics.

C. Random Forest

Random Forest is an ensemble of decision trees; each tree is trained on a random subset of the data and features, and the final prediction is the average of all trees. This averaging reduces variance and mitigates overfitting. The production configuration used 200 trees with a maximum depth of 12 and a fixed random seed for reproducibility. Random Forest naturally captures non-linear relationships and feature interactions and does not require feature scaling, since its splits depend only on thresholds. These properties make it well suited to the heterogeneous features in this project.

D. LSTM Neural Network

The Long Short-Term Memory (LSTM) network is a recurrent neural network designed for sequential data. Rather than treating each row independently, it processes a window of consecutive hourly observations and maintains an internal memory state across time steps. The model was implemented in PyTorch, with a single LSTM layer of 64 hidden units, a dropout of 0.3 for regularization, and a final linear layer mapping the last time step to the predicted AQI. It was trained on sequences of length 24 hours using the Adam optimizer (learning rate 0.005), mean-squared-error loss, a batch size of 32, and 150 epochs. PyTorch was chosen rather than TensorFlow because the available TensorFlow build required a version of the protobuf library incompatible with the Hopworks client; the project specification permits either framework for the advanced model, so this substitution was a sound engineering decision.

E. Results

Table I reports the performance of the three models on the 24-hour-ahead forecasting task.

TABLE I
MODEL PERFORMANCE (24-HOUR-AHEAD FORECAST)

Model	RMSE	MAE	R ²
Ridge Regression	20.31	13.76	0.103
Random Forest	19.30	14.01	0.190
LSTM	25.99	18.90	-0.465

Random Forest achieved the best performance on RMSE and R² and was selected as the production model. To verify that the model learns genuine structure rather than trivially repeating recent values, it was compared against a naive persistence baseline that predicts the future AQI to equal the current AQI; this baseline achieved an R² of 0.033, which Random Forest clearly exceeds.

A notable result is that the simpler Random Forest outperformed the more complex LSTM, which produced a negative R², indicating performance worse than predicting the mean. This is attributable to the relatively small dataset of roughly four thousand records: LSTMs have many parameters and typically require large volumes of data to generalize well, and on this dataset the network overfit despite dropout regularization. This finding illustrates an important lesson, that greater model

complexity does not guarantee better performance when data is limited.

Because the training pipeline retrains on continuously growing live data, the exact metric values vary between runs as the dataset expands; the figures reported here are drawn from the model-development dataset and are reproducible from the training notebook. The relative ranking of the models, however, remains stable: Random Forest consistently performs best, and the LSTM consistently underperforms on this data volume.

VI. EXPLAINABILITY WITH SHAP

To interpret the production model, SHAP (SHapley Additive exPlanations) was applied to the Random Forest using a tree explainer on a sample of the test set. SHAP attributes each prediction to its input features, giving both global importance and the direction of each feature’s effect. The analysis identified PM2.5 as the overwhelmingly dominant predictor, followed at a considerable distance by the 168-hour rolling AQI average, the cyclical month encoding, carbon monoxide, and temperature, with atmospheric pressure and the 24-hour PM2.5 lag also contributing. The directional (beeswarm) analysis confirmed physically sensible relationships: higher PM2.5 strongly increases the predicted AQI, while higher wind tends to reduce it. These results indicate that the model bases its predictions on meaningful physical factors rather than spurious correlations, supporting its trustworthiness.

VII. AUTOMATION WITH CI/CD

The feature and training pipelines were automated using GitHub Actions, a serverless CI/CD platform that is free for public repositories and integrates directly with the project’s repository. Two scheduled workflows were created. The feature pipeline workflow runs every hour to collect the latest reading and insert it into the Feature Store. The training pipeline workflow runs daily; it reads all accumulated features, repeats the feature engineering, experiments with all three models (Ridge Regression, Random Forest, and the LSTM), evaluates each using RMSE, MAE, and R², automatically selects the best model that produces a deployable artifact, and registers it in the Model Registry under the name `aqi_best_model`. In practice Random Forest is consistently selected. The LSTM training step is wrapped in defensive error handling so that, if the deep-learning dependency is unavailable or its step fails in the runner, the pipeline logs a warning and continues with the scikit-learn models rather than failing. Both workflows also support manual triggering, which was used during testing.

This design reflects a clean separation between experimentation and production: the exploratory notebook was used to develop and visualise the models in depth, while the automation script reproduces the full three-model comparison on a schedule and always promotes the empirically best model. As a result, the dashboard is always served by the freshest production model rather than a static, manually-saved artifact.

API credentials for AQICN and Hopworks are stored as encrypted repository secrets and injected into the workflows

as environment variables at runtime, so that no secret appears in the source code. Bringing the automated pipelines to a working state required iterative debugging of the cloud runner environment, which differs from the local development setup. Initial scheduled runs failed first because a pipeline script had not been committed to the repository, and subsequently because the Hopsworks client required additional components to write data to the Feature Store, specifically the Apache Arrow and Kafka client libraries. These issues were diagnosed from the workflow logs and resolved by committing the missing script and by installing the Hopsworks client with its full set of Python extras. Both workflows were then verified to complete successfully, and scheduled runs were confirmed to execute automatically, demonstrating that the system operates unattended. Consistent with documented platform behaviour, scheduled jobs on the free tier may occasionally be delayed under load and are disabled after a prolonged period of repository inactivity; neither affects the correctness of the implementation.

VIII. WEB APPLICATION AND DEPLOYMENT

The user-facing dashboard was built with Streamlit. On loading, it authenticates with Hopsworks and retrieves the production model `aqi_best_model`, together with its scaler and feature list, from the Model Registry, and the most recent engineered features from the Feature Store; these resources are cached to avoid repeated downloads.

The dashboard computes the three-day forecast using recursive multi-step prediction. The model first predicts the AQI 24 hours ahead. This prediction is then appended to the recent history as though it were an observed value, the features are recomputed, and the model predicts the following day; the process repeats for the third day. Implementing this required care with data types, as the synthetically appended rows could adopt an object data type and break the cyclical sine and cosine feature calculations; this was resolved by explicitly coercing the relevant columns to numeric values before recomputing features. The principal limitation of this technique is that prediction errors accumulate across steps, so the forecast for later days is less reliable; this limitation is disclosed directly on the dashboard. The dashboard queries the Model Registry for the latest version of the production model, so that it always serves the most recently retrained model, with a fallback to a known version should the lookup fail.

The interface presents the current AQI with its health category, a colour-coded three-day forecast in which the colour corresponds to the standard US EPA health categories, an interactive chart showing the recent measured trend alongside the forecast, and an automated alert. The alert evaluates the maximum forecast AQI and, if any day exceeds the unhealthy threshold of 150, displays a prominent warning identifying the affected day and advising sensitive groups to limit outdoor activity; otherwise it confirms that no hazardous levels are expected.

The application was deployed publicly on Streamlit Community Cloud, linked to the project repository, with the

Hopsworks credential supplied through Streamlit’s encrypted secrets mechanism. Because it reads from the continuously updated Feature Store and Model Registry, the dashboard automatically reflects the latest data and model produced by the automated pipelines.

IX. LIMITATIONS AND FUTURE WORK

The project meets its objectives but has several honest limitations. First, the historical dataset spans approximately six months; a longer history of one to two years would likely improve all models and would particularly benefit the data-hungry LSTM. Second, multi-day forecasting accuracy is fundamentally constrained because future AQI depends on future weather that is unknowable from present data; incorporating weather *forecast* data as input features is the most promising direction for substantial improvement. Third, the recursive forecasting strategy accumulates error across days, so later days are inherently less accurate. Fourth, AQI is computed from PM2.5 alone rather than as the maximum across all pollutant sub-indices. Finally, the system models a single city. Future work could expand the dataset, integrate forecast weather, compute a full multi-pollutant AQI, support multiple cities, and revisit sequence models once sufficient data is available.

X. CONCLUSION

The Pearls AQI Predictor is a complete, automated, serverless machine learning system that forecasts Karachi’s air quality for the next three days. It collects data automatically, stores features and models in a managed registry, retrains on a schedule, and serves predictions through a publicly accessible dashboard with health alerts. The project fulfils all core requirements: an end-to-end prediction system, a scalable automated pipeline, an interactive dashboard presenting real-time and forecasted data, and thorough documentation. Beyond the implementation, the project demonstrates sound engineering judgment, including the diagnosis and correction of a data-quality problem, the resolution of dependency conflicts in the automation environment, the empirical selection of a model based on evidence rather than complexity, a clean separation between experimentation and production, and the honest presentation of results and limitations, including the genuine difficulty of forecasting air quality several days in advance.